دانشگاه صنعتی شریف

گزارش نهایی پروژه طراحی و پیاده‌سازی سیستم بینایی ماشین و تشخیص الگو

(با رویکرد پردازش موازی در معماری x86 و دستورات SIMD)

نام و نام خانوادگی: حامد علی نژاد

شماره دانشجویی: 403106335

استاد درس: دکتر امیرحسین جهانگیر

فصل اول: مقدمه و معرفی پروژه

۱-۱. بیان مسئله و اهمیت موضوع

در دهه‌های اخیر، پردازش تصویر و بینایی ماشین به یکی از ارکان اصلی سیستم‌های هوشمند، از تشخیص چهره و پایش ترافیک گرفته تا تشخیص‌های پزشکی و صنعتی، تبدیل شده است. قلب تپنده‌ی بسیاری از این الگوریتم‌ها، از جمله فیلترهای تشخیص لبه و شبکه‌های عصبی پیچشی (CNN)، عملیات ریاضی «**کانولوشن**» (**Convolution**) و «**همبستگی متقابل**» (**Cross-Correlation**) است.

با این حال، اجرای این عملیات بر روی تصاویر با رزولوشن بالا نیازمند انجام میلیون‌ها محاسبه‌ی ضرب و جمع اعشاری در ثانیه است. پیاده‌سازی این الگوریتم‌ها با زبان‌های سطح بالا (مانند پایتون) یا حتی پیاده‌سازی‌های استاندارد در زبان C، به دلیل ایجاد گلوگاه‌های محاسباتی و عدم استفاده بهینه از معماری سخت‌افزار، زمان اجرای بالایی دارند. از این رو، استفاده از برنامه‌نویسی سطح پایین (زبان اسمبلی) و بهره‌گیری از قابلیت‌های پردازش موازی درون‌پردازنده‌ای، یک راهکار مهندسی ضروری برای دستیابی به عملکرد بلادرنگ (Real-time) محسوب می‌شود.

۱-۲. اهداف پروژه

هدف اصلی این پروژه، طراحی یک سیستم کامل و چندلایه برای تشخیص الگوهای تصویری است؛ به‌طوری که تفاوت سطح عملکرد (Performance) پردازنده در اجرای کدهای مقیاس‌پذیر بررسی شود. در این راستا، اهداف زیر دنبال شده‌اند:

- پیاده‌سازی الگوریتم پایه کانولوشن دو بعدی با زبان خالص C جهت ایجاد یک مبنای مقایسه (Baseline).
- پیاده‌سازی بهینه‌سازی‌شده‌ی همان الگوریتم در زبان اسمبلی معماری x86 با استفاده از رجیسترهای ۱۲۸-بیتی XMM و دستورات **SIMD (Single Instruction, Multiple Data)** جهت پردازش موازی پیکسل‌ها.

- حل چالش‌های سطح پایینِ مدیریت حافظه (تخصیص حافظه پیوسته) و تطابق سیستم‌های مختصات (Bottom-Up در فرمت فایل‌های BMP و Top-Down در ماتریس‌های ریاضی).
- تحلیل آماری و بنچمارک دقیق زمان اجرای الگوریتم‌ها بر روی یک مجموعه داده (Dataset) استاندارد.
- ارتقای سیستم از یک لبه‌یاب ساده به یک طبقه‌بند ارقام دست‌نویس (Digit Classifier) با الگوبرداری از دادگان MNIST و استفاده از تکنیک تطبیق الگو (Template Matching).

۱-۳. ساختار گزارش

این گزارش به گونه‌ای تدوین شده است که مسیر تکامل پروژه را از مفاهیم پایه تا استنتاج نهایی به تصویر بکشد:

- **فصل دوم (معماری و پیاده‌سازی پایه)**: به بررسی نحوه تولید داده‌ها، مدیریت فرمت تصاویر و پیاده‌سازی کانولوشن در زبان C می‌پردازد.
- **فصل سوم (بهینه‌سازی سخت‌افزاری)**: جزئیات انتقال محاسبات به زبان اسمبلی و استفاده از تکنیک موازی‌سازی SIMD را تشریح می‌کند.
- **فصل چهارم (تحلیل آماری و مصورسازی)**: نتایج بنچمارک و نمودارهای مقایسه زمان اجرا (Speedup) را ارائه می‌دهد.
- **فصل پنجم (تشخیص ارقام - بخش توسعه‌یافته)**: نحوه استفاده از موتور کانولوشن توسعه‌یافته برای طبقه‌بندی هوشمند اعداد (تطبیق الگو) را بررسی کرده و در نهایت گزارش را نتیجه‌گیری می‌کند.

فصل دوم: معماری سیستم، پردازش تصویر و پیاده‌سازی کانولوشن

۲-۱. ساختار داده‌ها و خواندن فایل‌های تصویری (فرمت BMP)

پیش از اعمال هرگونه الگوریتم پردازشی، اولین چالش در سیستم‌های بینایی ماشین، استخراج صحیح داده‌های پیکسلی از فایل‌های تصویری است. در این پروژه برای پرهیز از وابستگی به کتابخانه‌های سطح بالای خارجی (مانند OpenCV در زبان C) و کنترل دقیق‌تر روی حافظه، تصاویر با فرمت استاندارد و غیرفشرده‌ی BMP (Bitmap) مورد استفاده قرار گرفتند.

فرمت BMP شامل یک هدر (Header) ۵۴ بایتی است که اطلاعات حیاتی تصویر از جمله عرض، ارتفاع و آفستِ شروعِ داده‌های پیکسلی را در خود جای داده است. در نوشتن تابع `read_bmp_color` در زبان C، دو چالش مهم وجود داشت که با موفقیت برطرف گردید:

1. **پدینگ سطرها (Row Padding)**: در استاندارد BMP، طول هر سطر از پیکسل‌ها در حافظه باید مضربی از ۴ بایت باشد. برای تصاویر رنگی (۳ کانال رنگی RGB)، فرمول `(width * 3 + 3) & (~3)` جهت محاسبه دقیق پدینگ و پرش از روی بایت‌های خالی پیاده‌سازی شد تا از شیفت‌خوردگی رنگ‌ها در تصویر جلوگیری شود.
2. **سیستم مختصات معکوس (Bottom-Up)**: برگرداندن و اصلاح مختصات Y. برخلاف ماتریس‌های استاندارد ریاضی که مبدأ مختصات در بالا-سمت چپ قرار دارد، پیکسل‌ها در فایل‌های BMP از پایین به بالا ذخیره می‌شوند. این تفاوت در اندیس‌گذاری آرایه‌های یک‌بعدیِ تخصیص‌یافته در حافظه داینامیک (`malloc`) لحاظ گردید.

۲-۲. مبانی ریاضی الگوریتم کانولوشن دو بعدی

عملیات کانولوشن (Convolution) پایه و اساس بسیاری از الگوریتم‌های استخراج ویژگی (Feature Extraction) است. در این پروژه، برای تشخیص لبه‌ها و یافتن مکان اشکال هندسی، از یک کرنل لاپلاس (Laplacian Kernel) با ابعاد ۳ در ۳ استفاده شد.

از منظر ریاضی، کانولوشن گسسته‌ی یک تصویر I با یک کرنل K (با ابعاد 2m+1 در 2m+1) برای محاسبه مقدار پیکسل خروجی در مختصات(x,y) به شکل زیر تعریف می‌شود:

$$O(x,y)=\sum_{i=-m}^{m}\sum_{j=-m}^{m} I(x+i,y+j)\cdot K(i,j)$$

این فرمول نشان می‌دهد که برای محاسبه‌ی تنها یک پیکسل در تصویر خروجی، نیاز به انجام ۹ عمل ضرب و ۸ عمل جمع اعشاری است.

۲-۳. پیاده‌سازی پایه در زبان C (رویکرد اسکالر)

برای ایجاد یک مبنای مقایسه (Baseline)، الگوریتم فوق ابتدا به صورت خالص در زبان C پیاده‌سازی شد. تابع convolution_pure_c با استفاده از حلقه‌های تو در تو (Nested Loops)، کرنل ۳ در ۳ را بر روی تک‌تک پیکسل‌های تصویر می‌لغزاند.

پیچیدگی زمانی این الگوریتم از مرتبه O(W*H*K^2) است (که در آن W و H ابعاد تصویر و K ابعاد کرنل است). پردازنده در اجرای این کد از رویکرد پردازش اسکالر (Scalar Processing) استفاده می‌کند؛ به این معنا که در هر چرخه ساعت (Clock Cycle)، تنها یک عمل ضرب یا جمع روی یک جفت عدد انجام می‌شود. این رویکرد، به خصوص برای تصاویری با رزولوشن بالا (مانند ۵۰۰ در ۵۰۰ پیکسل)، باعث ایجاد یک گلوگاه محاسباتی (Computational Bottleneck) شدید شده و زمان اجرای برنامه را به شدت افزایش می‌دهد.

۲-۴. بهینه‌سازی سخت‌افزاری با زبان اسمبلی و پردازش موازی (SIMD)

برای رفع گلوگاه محاسباتی فاز قبل، بخش کانولوشن به زبان اسمبلی x86 منتقل شد. ایده اصلی در این بخش، گذر از پردازش اسکالر و بهره‌گیری از توازی در سطح داده یا Data-Level Parallelism (DLP) بود.

برای این منظور، از معماری و دستورات SIMD (Single Instruction, Multiple Data) در پردازنده‌های مدرن استفاده گردید.

در این پیاده‌سازی، به جای استفاده از رجیسترهای عمومی ۳۲ یا ۶۴ بیتی، از رجیسترهای ۱۲۸-بیتیِ XMM استفاده شد. هر رجیستر XMM توانایی گنجایش ۴ عدد اعشاری ۳۲-بیتی (Single Precision Float) را به صورت همزمان دارد.

تکنیک‌های کلیدی استفاده شده در تابع `apply_convolution_asm` عبارتند از:

- **بارگذاری موازی**: استفاده از دستورات `movups` برای خواندن همزمان ۴ پیکسل از حافظه اصلی و انتقال آن به رجیستر XMM.
- **محاسبات برداری**: جایگزینی دستورات ضرب و جمع معمولی با دستورات `mulps` (Multiply Packed Single-precision) و `addps` (Add Packed Single-precision). این دستورات با اجرای تنها یک دستورالعمل در پردازنده، ۴ عمل ضرب یا جمع را به صورت موازی روی ۴ جفت عدد انجام می‌دهند.
- **کاهش سربار حلقه‌ها**: با پردازش ۴ پیکسل در هر تکرار، تعداد پرش‌های شرطی (Branching) در کدهای اسمبلی به یک چهارم کاهش یافت که این امر به بهبود عملکرد پیش‌بینی‌کننده پرش (Branch Predictor) در لوله‌کشی (Pipeline) پردازنده کمک ویژه ای کرد.

۲-۵. ارزیابی صحت عملکرد الگوریتم بر روی تصویر استاندارد لِنا (Lena)

در ادبیات پردازش تصویر، استفاده از تصاویر Benchmark برای ارزیابی کیفی و بصری الگوریتم‌ها یک رویه کاملاً استاندارد است. تصویر «لِنا» (Lena) به دلیل دارا بودن ترکیبی ایده‌آل از بافت‌های پیچیده (مانند پرهای کلاه)، نواحی هموار (پوست)، لبه‌های تیز و طیف گسترده‌ای از سایه‌روشن‌ها، از دهه ۱۹۷۰ تاکنون مهم‌ترین تصویر تست در این حوزه محسوب می‌شود.

در این فاز از پروژه، برای اطمینان از صحت محاسبات ریاضی و مدیریت درست کانال‌های رنگی (RGB) در هر دو موتور پردازشی C و اسمبلی، تصویر رنگی لِنا به عنوان ورودی سیستم انتخاب شد. به منظور بررسی جامع‌تر، موتور کانولوشن با سه کرنل (Kernel) معروف و پرکاربرد زیر مورد آزمایش قرار گرفت:

نحوه اجرای فایل ها:

```
gcc just_c.c -o just_c -no-pie -O2      ./just_c ,
nasm -f elf64 calc.asm -o calc.o
gcc main.c calc.o -o main -no-pie -O2   ./main
```

۱. **فیلتر تشخیص لبه (Edge Detection)**: برای این منظور از کرنل لاپلاس (Laplacian) استفاده شد. این کرنل (با مرکزیت مقدار ۴ و همسایه‌های بالا، پایین، چپ و راست برابر با ۱-) با محاسبه مشتق مکانیِ شدت روشنایی، تغییرات ناگهانی در تصویر را شناسایی می‌کند. با اعمال این کرنل بر روی تصویر لِنا، تمامی نواحی هموار (تغییرات صفر) به رنگ سیاه درآمده و خطوط و مرزهای اشیاء (مانند فرم صورت و کلاه) به رنگ روشن برجسته شدند.

۲. فیلتر محوکننده (Box Blur): این فیلتر به عنوان یک فیلتر پایین‌گذر (Low-pass Filter) عمل می‌کند. با استفاده از یک ماتریس ۳ در ۳ که تمام درایه‌های آن مقداری برابر با کسر ۱/۹ (یک نهم) دارند، مقدار هر پیکسل با میانگین همسایگانش جایگزین شد. خروجی این کرنل، تصویری نرم‌تر بود که در آن نویزهای فرکانس بالا حذف شده بودند.

۳. فیلتر وضوح‌بخش (Sharpen): این کرنل برعکس فیلتر محوکننده عمل می‌کند. با تخصیص یک وزن مثبت بزرگ به پیکسل مرکزی (مثلاً ۵) و وزن‌های منفی (مثلاً ۱-) به همسایه‌های آن، تفاوت بین پیکسل‌ها تشدید می‌شود. با اعمال این کرنل، جزئیات بافت‌ها و کنتراست تصویر لِنا به شکل چشم‌گیری افزایش یافت.

نتایج به‌دست‌آمده: خروجی‌های تولید شده از اجرای برنامه با زبان C خالص و نسخه بهینه‌شده با دستورات SIMD اسمبلی، از نظر پیکسلی کاملاً یکسان (Bit-exact) بودند. این تطابق ۱۰۰ درصدی ثابت کرد که منطق ریاضی الگوریتم، پرش از روی پدینگ‌ها (Row Padding) و مدیریت سرریز اعداد (با محدود کردن مقادیر نهایی در بازه ۰ تا ۲۵۵) در کدهای سطح پایین کاملاً بدون باگ پیاده‌سازی شده است.

نتیجه‌ی این مهندسی در سطح پایین، تبدیل یک الگوریتم به شدت کند به یک موتور پردازش تصویرِ بلادرنگ (Real-time) بود که توانست راه را برای پیاده‌سازی فازهای پیشرفته‌تر (مانند دسته‌بندی اشکال و اعداد) هموار سازد.

فصل سوم: تشخیص پیشرفته، مصورسازی و تحلیل عملکرد سیستم

۳-۱. ارتقای سیستم به تشخیص چندگانه اشکال هندسی

پس از موفقیت در مکان‌یابی اشیاء ساده، سیستم برای طبقه‌بندی (Classification) اشکال هندسی مختلف ارتقا یافت. در این فاز، ترکیب استخراج ویژگی‌های سطح پایین (لبه‌ها توسط اسمبلی) و ویژگی‌های سطح بالا (مساحت شیء) مورد استفاده قرار گرفت.

الگوریتم توسعه‌یافته در زبان C، پس از دریافت نقشه لبه‌ها، با پیمایش پیکسل‌های تصویر اصلی اقدام به محاسبه مساحت (تعداد پیکسل‌های سفید) هر شیء می‌کند. با توجه به تفاوت معنادار مساحت در ابعاد برابر (به عنوان مثال، مساحت مربع بیشتر از دایره و دایره بیشتر از مثلث است)، سیستم توانست با دقت بالایی اشکال را از یکدیگر تفکیک کند.

نتایج خروجی در قالب یک فایل CSV ذخیره شده و سپس توسط یک اسکریپت پایتون، کادرهای رنگی متفاوتی (قرمز برای مربع، آبی برای دایره و سبز برای مثلث) به همراه برچسب متنی بر روی تصاویر رسم گردید. در نهایت، اتوماسیون پایتون تمامی فریم‌های پردازش‌شده را به یک انیمیشن (GIF) تبدیل کرد تا درستی عملکرد سیستم به صورت بصری قابل ارزیابی باشد.

نحوه اجرای فایل ها:

۳-۲. مینی‌پروژه: طبقه‌بندی ارقام دست‌نویس (MNIST Dataset)

برای اثبات مقیاس‌پذیری و کاربردی بودن موتور پردازشی توسعه‌یافته در مسائل واقعیِ بینایی ماشین، یک سیستم تشخیص ارقام دست‌نویس مبتنی بر دادگان استاندارد MNIST طراحی و پیاده‌سازی شد. این دادگان شامل تصاویری با ابعاد ۲۸ در ۲۸ پیکسل (مجموعاً ۷۸۴ پیکسل) از اعداد ۰ تا ۹ است.

در این فاز، به جای استفاده از شبکه‌های عصبی پیچیده، از تکنیک تطبیق الگو (Template Matching) مبتنی بر محاسبه خطای میانگین مربعات (Mean Squared Error) استفاده گردید. در فاز آموزش (

Training)، میانگین تمامی تصاویر مربوط به هر رقم محاسبه شد تا ۱۰ الگوی مرجع (Template) استخراج شود.

برای تشخیص رقم یک تصویر جدید، فاصله اقلیدسی مربع‌یافته (Squared Euclidean Distance) بین تصویر ورودی و هر ۱۰ الگو محاسبه می‌شود. فرمول ریاضی این عملیات به شرح زیر است:

$$D=\sum_{i=1}^{784}\left(I_{test}[i]-I_{template}[i]\right)^2$$

هر الگویی که کمترین فاصله (خطا) را با تصویر تست داشته باشد، به عنوان رقم پیش‌بینی‌شده انتخاب می‌گردد.

۳-۳. ارتقا به معماری AVX و پردازش موازی ۲۵۶-بیتی

با توجه به اینکه محاسبه فاصله برای ۱۰۰۰۰ تصویر تست، نیازمند ده‌ها میلیون عملیات تفریق، به توان دو رساندن و جمع اعشاری است، کد اسمبلی به معماری پیشرفته‌تر Advanced Vector Extensions (AVX) ارتقا یافت.

در این پیاده‌سازی جدید، از رجیسترهای ۲۵۶-بیتی YMM استفاده شد. این رجیسترها ظرفیت نگهداری ۸ عدد اعشاری ۳۲-بیتی را به صورت همزمان دارند. الگوریتم اسمبلی (calculate_distance_simd) با استفاده از دستورات vmovups (بارگذاری)، vsubps (تفریق موازی)، vmulps (ضرب موازی برای توان دو) و vaddps (جمع موازی)، در هر چرخه اجرای حلقه، ۸ پیکسل را پردازش می‌کند. در نتیجه، پردازش کل یک تصویر ۷۸۴ پیکسلی، تنها در ۹۸ تکرار حلقه (RCX = 98) به اتمام می‌رسد که نشان‌دهنده بهینه‌سازی حداکثری در سطح سخت‌افزار است.

۳-۴. تحلیل آماری و بنچمارک عملکرد (Benchmark Results)

برای ارزیابی دقیق عملکرد، کدهای C و اسمبلی در قالب یک کتابخانه اشتراکی (libclassifier.so) کامپایل شده و از طریق واسط ctypes مستقیماً به هسته پایتون متصل شدند. این معماری دقیقا مشابه سازوکار کتابخانه‌های مطرح هوش مصنوعی نظیر TensorFlow است.

یک بنچمارک جامع بر روی تمامی ۱۰۰۰۰ تصویر تستِ دادگان MNIST انجام شد و زمان اجرای الگوریتم طبقه‌بندی در دو حالت زیر با یکدیگر مقایسه گردید:

۱. اجرای خالص در پایتون با بهره‌گیری از توابع برداری کتابخانه NumPy.

۲. اجرای ترکیبی C و اسمبلی با دستورات SIMD (256-bit).

نتایج به‌دست‌آمده:

- **دقت تشخیص**: هر دو پیاده‌سازی دقت کاملاً یکسانی را ثبت کردند که نشان از صحت منطق ریاضی در کد اسمبلی دارد.
- **سرعت اجرا**: موتور پردازشی اسمبلی موفق شد زمان اجرای عملیات را به شدت کاهش دهد. با توجه به خروجی‌های ثبت شده، معماری C/ASM توانست به تسریع چشمگیری (Speedup) نسبت به پیاده‌سازی پایتون/NumPy دست یابد. غلبه بر سرعت کتابخانه قدرتمندی چون NumPy (که خود در پس‌زمینه با زبان C نوشته شده است)، ارزش بالای برنامه‌نویسی سطح پایین و استفاده از رجیسترهای YMM را در گلوگاه‌های محاسباتی اثبات می‌کند.

این نتایج در قالب نمودارهای میله‌ای مصورسازی شدند تا مقایسه زمان‌های اجرا و میزان بهبود عملکرد به روشنی قابل استناد باشد.

نحوه اجرای مینی پروژه:

ابتدا در ترمیل دستور make را نوشته تا libclassifier.so ساخته شود. سپس با python3 main.py پروژه را اجزا میکنیم.

فصل چهارم (بخش امتیازی): تشخیص تومور مغزی با شبکه عصبی پیچشی (CNN)

۴-۱. مقدمه و چالش مسئله

در سال‌های اخیر، شبکه‌های عصبی پیچشی (Convolutional Neural Networks) به استاندارد طلایی در تحلیل تصاویر پزشکی تبدیل شده‌اند. با این حال، اجرای این شبکه‌ها معمولاً به سخت‌افزارهای قدرتمند و کتابخانه‌های سطح بالا (نظیر TensorFlow یا PyTorch) وابستگی دارد. هدف از این بخش امتیازی، اثبات امکان‌پذیریِ اجرای یک مدل هوش مصنوعی در لبه (Edge Computing) با توسعه‌ی یک موتور استنتاج (Inference Engine) سفارشی در زبان C بود. مجموعه داده (Dataset) انتخاب شده برای این فاز، تصاویر MRI مغز از پایگاه داده Kaggle است که هدف آن، طبقه‌بندی دوتایی (Binary Classification) تصاویر به دو کلاس «سالم» (No Tumor) و «دارای تومور گلیوما» (Glioma) می‌باشد.

۴-۲. پیش‌پردازش و معماری شبکه (Micro CNN)

به منظور امکان‌پذیری پیاده‌سازی شبکه از پایه (From Scratch) در زبان‌های سطح پایین، تصاویر MRI ابتدا به حالت سیاه‌وسفید (Grayscale) تک‌کاناله تبدیل و به ابعاد استاندارد ۶۴ در ۶۴ پیکسل تغییر اندازه یافتند تا بار محاسباتی بهینه‌سازی شود. سپس یک معماری سبک و مینیمال (Micro CNN) طراحی گردید که شامل لایه‌های زیر است:

1. **لایه کانولوشن (Conv2D)**: استفاده از ۴ فیلتر (کرنل) با ابعاد ۳ در ۳ جهت استخراج ویژگی‌های محلی (Local Features) مانند بافت و لبه‌های تومور. خروجی این لایه از تابع فعال‌ساز ReLU عبور می‌کند تا مقادیر منفی (ویژگی‌های بی‌اثر) صفر شوند.

2. **لایه ادغام (Max Pooling)**: با ابعاد ۲ در ۲ برای کاهش ابعاد تصویر (از ۶۲x۶۲ به ۳۱x۳۱) و حفظ مهم‌ترین ویژگی‌های استخراج‌شده، که به کاهش حساسیت مدل به جابجایی مکانی تومور کمک می‌کند.

3. **لایه تمام‌متصل (Dense / Fully Connected)**: تبدیل ماتریس سه‌بعدی خروجی به یک بردار مسطح (Flatten) به طول ۳۸۴۴ (۳۱× ۳۱× ۴) و اتصال آن به یک نورون خروجی.

۴-۳. فاز آموزش و استخراج پارامترها (Weights Extraction)

از آنجا که الگوریتم پس‌انتشار خطا (Backpropagation) و محاسبه گرادیان‌ها در فاز آموزش نیازمند توان پردازشی بسیار بالایی است، این فاز با استفاده از زبان پایتون انجام شد. پس از همگرایی مدل و رسیدن به دقت مطلوب در دسته‌بندی تصاویر، یک اسکریپت اختصاصی برای استخراج پارامترهای یادگرفته‌شده نوشته شد. در این مرحله، ماتریس وزن‌های لایه کانولوشن، بایاس‌ها (Biases) و تمامی ۳۸۴۴ وزن لایه نهایی در قالب فایل‌های متنی استاندارد استخراج شدند تا به عنوان «مغزِ آموزش‌دیده» در اختیار موتور C قرار گیرند.

۴-۴. توسعه موتور استنتاج (Inference) در زبان C

هسته اصلی این بخش امتیازی، پیاده‌سازی گام‌به‌گام (Forward Pass) شبکه عصبی در زبان C بدون وابستگی به هیچ کتابخانه خارجی است. این موتور وظایف زیر را بر عهده دارد:

- **بارگذاری ساختاریافته**: خواندن ماتریس پیکسل‌های تصویر ورودی و لود کردن وزن‌های استخراج‌شده از فاز قبل در حافظه.
- **اجرای لایه کانولوشن**: لغزاندن ۴ فیلتر مختلف بر روی تصویر با استفاده از حلقه‌های تو در تو و اعمال تابع شرطی ReLU بر روی حاصل‌جمع‌ها.

•**اجرای تابع فعال‌ساز نهایی (Sigmoid)**: پس از محاسبه ضرب داخلی (Dot Product) در لایه Dense، خروجی نهایی باید به یک احتمال (درصد اطمینان بین ۰ و ۱) تبدیل شود. برای این کار، تابع سیگموید در زبان C پیاده‌سازی شد که فرمول ریاضی آن به شرح زیر است:

$$Sigmoid(x)=\frac{1}{1+e^{-x}}$$

•**تصمیم‌گیری**: اگر خروجی تابع سیگموید بزرگتر از ۰.۵ باشد، سیستم وجود تومور را اعلام کرده و در غیر این صورت بافت را سالم تشخیص می‌دهد.

۵-۴. ارزیابی خروجی

با اتصال این موتور C به تصاویر واقعی تست، سیستم توانست با موفقیت و در کسری از ثانیه، ماتریس‌های ورودی را تحلیل کرده و گزارش تشخیص (Diagnosis Report) را به همراه «درصد اطمینان» (Confidence Score) در خروجی استاندارد چاپ نماید. این دستاورد نشان می‌دهد که توابع کانولوشن پیاده‌سازی شده در فازهای پیشین، نه تنها برای شناسایی اشکال ساده، بلکه به عنوان هسته اصلی سیستم‌های پیشرفته هوش مصنوعی در تجهیزات پزشکی قابل استفاده هستند.

فصل پنجم: نتیجه‌گیری و جمع‌بندی

این پروژه، یک مسیر تکاملی جامع از مفاهیم پایه‌ای پردازش تصویر تا پیاده‌سازی سیستم‌های مدرن هوش مصنوعی را به تصویر کشید. هدف نهایی، صرفاً شناسایی یک الگو یا شیء در تصویر نبود؛ بلکه درک عمیقِ گلوگاه‌های محاسباتی (Computational Bottlenecks) و غلبه بر آن‌ها با استفاده از معماری سخت‌افزار بود.

دستاوردهای کلیدی این پروژه را می‌توان در سه محور اصلی خلاصه کرد:

۱. **اهمیت درک سخت‌افزار و پردازش موازی (SIMD)**: پیاده‌سازی الگوریتم‌های کانولوشن و محاسبه فاصله اقلیدسی (برای دادگان MNIST) نشان داد که زبان C، با وجود سرعت بالا نسبت به زبان‌های تفسیری، در برابر حجم عظیم داده‌های تصویری دچار محدودیت پردازش اسکالر می‌شود. با انتقال هسته محاسباتی به زبان اسمبلی x86 و بهره‌گیری از رجیسترهای ۱۲۸-بیتی (SSE) و ۲۵۶-بیتی (AVX)، توانستیم موازی‌سازی در سطح داده (DLP) را محقق کنیم. نتایج بنچمارک‌ها و تسریعِ ده‌ها برابری (Speedup) اثبات کرد که برای دستیابی به عملکرد بلادرنگ (Real-time) در سیستم‌های بینایی ماشین، استفاده از دستورات SIMD یک الزام مهندسی است، نه صرفاً یک انتخاب.

۲. **هم‌افزایی زبان‌های برنامه‌نویسی (Heterogeneous Architecture)**: یکی از بزرگترین موفقیت‌های این سیستم، ایجاد یک معماری ترکیبی و بهینه بود. در این رویکرد، زبان پایتون به دلیل داشتن کتابخانه‌های غنی، وظیفه تولید دادگان، آموزش شبکه‌های عصبی (Training) و مصورسازی آماری را بر عهده گرفت. در مقابل، زبان C برای مدیریت دقیق حافظه (خواندن فرمت BMP) و زبان اسمبلی به عنوان موتور محرک محاسبات سنگین (Inference Engine) عمل کردند. این معماری دقیقاً مشابه ساختار فریم‌ورک‌های صنعتیِ روز دنیا نظیر TensorFlow است.

۳. **مقیاس‌پذیری از لبه‌یابی تا یادگیری عمیق (Deep Learning)**: روند پروژه ثابت کرد که مفاهیم پایه‌ای مانند فیلتر لاپلاس و تطبیق الگو (Template Matching)، آجرهای سازنده‌ی سیستم‌های بسیار پیچیده‌تر هستند. موتوری که در ابتدا برای یافتن یک مربع ساده طراحی شده بود، با ارتقای منطق و تزریق

وزن‌های از پیش‌آموزش‌دیده، توانست ارقام دست‌نویس را دسته‌بندی کرده و در بخش امتیازی، به عنوان یک شبکه عصبی پیچشی (CNN) کاملاً مستقل برای تشخیص تومور مغزی از روی تصاویر MRI ایفای نقش کند.

سخن پایانی: توسعه نرم‌افزار در پایین‌ترین سطوح سخت‌افزاری (Bare-metal Optimization) و ترکیب آن با الگوریتم‌های هوش مصنوعی، نشان داد که با درک درست از معماری کامپیوتر، می‌توان شبکه‌های پیچیده را حتی در محیط‌هایی با منابع محدود (Edge Devices) و بدون نیاز به کتابخانه‌های سنگین شخص ثالث، با بالاترین سرعت و دقت اجرا نمود.

با تشکر از استاد جهانگیر بابت معرفی این پروژه

حامد علی نژاد

1404/12/05